

University of Pisa
M.Sc. in Data Science and
Business Informatics



Optimization for Data Science

Quadratic Optimization on Simplices:
An Implementation of the Frank-Wolfe Algorithm

P-35 members:

Camilla Chiruzzi
c.chiruzzi@studenti.unipi.it, Matricola: 665334

Niccolò Seghieri
n.seghieri@studenti.unipi.it, Matricola: 666410

Academic Year 2024/2025

Contents

1	Setting the stage	2
1.1	The problem	2
1.2	The Algorithms	2
1.2.1	Algorithm A1: Conditional Gradient (Frank-Wolfe)	2
1.2.2	Algorithm A2: general-purpose solver	2
2	Function properties	2
3	Frank-Wolfe algorithm	3
3.1	General Frank-Wolfe algorithm	4
3.2	Frank-Wolfe algorithm over disjoint simplices	4
3.3	The Frank-Wolfe gap	7
3.4	Step size selection	7
3.5	Away-Step Frank-Wolfe	9
3.5.1	General Away-Step Frank-Wolfe	9
3.5.2	Away-Step Frank-Wolfe algorithm over disjoint simplices	10
4	Convergence analysis	13
4.1	Convergence analysis for Vanilla Frank-Wolfe	13
4.2	Convergence analysis for Away-Step Frank-Wolfe	14
4.3	Constants estimation	16
5	Experiments	18
5.1	Instance generation and initial point selection	18
5.2	q - generation strategy: results comparison	22
5.3	Impact of the dimensionality	23
5.4	Impact of the spectral radius	26
5.5	Impact of the Null space	28
5.6	General purpose solver Gurobi	31
5.6.1	Standard configuration	31
5.6.2	Ill-conditioned configuration	31
5.7	General discussion	32

1 Setting the stage

1.1 The problem

The problem to be solved is a convex quadratic program on a set of disjoint simplices using an algorithm of the class of conditional gradient (a.k.a. Frank-Wolfe type) methods. Specifically, the objective is to minimize the quadratic function $f : \mathcal{D} \subseteq \mathbb{R}^n \rightarrow \mathbb{R}$ of the form:

$$f(x) = x^T Q x + q x, \quad (1)$$

where $Q \in \mathbb{R}^{n \times n}$ is a positive semidefinite symmetric matrix and $x, q \in \mathbb{R}^n$. The constrained domain $\mathcal{D}$ is defined as follows:

$$\mathcal{D} = \{x \in \mathbb{R}^n \mid \sum_{i \in I^k} x_i = 1 \quad \forall k \in K, \quad x \geq 0\}. \quad (2)$$

where the index sets I^k form a partition of $\{1, \dots, n\}$, and each simplex corresponds to one of these sets. So, $\bigcup_{k \in K} I^k = \{1, \dots, n\}$ and $I^h \cap I^k = \emptyset$.

So the problem (P) can be written in a more compact ways as:

$$\min \left\{ x^T Q x + q x : \sum_{i \in I^k} x_i = 1 \quad \forall k \in K, \quad x \geq 0 \right\}. \quad (3)$$

The goal is to find a solution x^* that minimizes the objective function subject to the specified constraints.

1.2 The Algorithms

Two distinct algorithms are considered to solve this problem:

1.2.1 Algorithm A1: Conditional Gradient (Frank-Wolfe)

Algorithm A1 belongs to the class of conditional gradient methods (Frank-Wolfe type). This algorithm iteratively solves a linear approximation of the objective function, which is particularly useful when the feasible region is defined by a set of disjoint simplices. A modified version of this algorithm will be applied to efficiently handle the structure of the problem, especially the partitioned nature of the constraint set.

If necessary, the algorithm will be adapted to include an exact line search for the step size, taking advantage of the quadratic form of the problem.

1.2.2 Algorithm A2: general-purpose solver

Additionally, a general-purpose solver will be used to handle the problem in a standard formulation. This solver will deal with the overall structure of the problem and provide a benchmark for comparison. It is important to note that the linear programming subproblem in A1 will be solved using a tailored approach that exploits the special structure of the problem. No off-the-shelf solvers will be directly applied to the main optimization problem.

2 Function properties

A preliminary study should be carried out concerning the properties and conditions of the function and its constraints in order to be able to use the Frank - Wolfe algorithm which

requires convexity.

A function $f : \mathcal{D} \subseteq \mathbb{R}^n \rightarrow \mathbb{R}$ is defined to be convex if [5]:

$$\forall (x, y, \theta) \in \mathcal{D} \times \mathcal{D} \times [0, 1], f((1 - \theta)x + \theta y) \leq (1 - \theta)f(x) + \theta f(y).$$

The function (1) is therefore definitely convex, since the sum of two convex functions preserves convexity: the quadratic form is in fact convex since the matrix Q is a positive semidefinite symmetric matrix and the function qx is convex (and at the same time concave) as it is a linear function.

Analysing the domain (2) it can be seen that each partition is represented by a standard simplex, i.e. $\sum_{i \in I^k} x_i = 1 \quad \forall k \in K, \quad x \geq 0$, and I^k defines the polytope in $\mathbb{R}^n$ as a convex hull of its vertices V^k .

The domain $\mathcal{D}$ can be defined as the cartesian product of K simplices, which preserves convexity:

$$\mathcal{D} = \prod_{k=1}^K P^k = P^1 \times P^2 \times \dots \times P^K \quad (4)$$

Since each P^k is a convex set, and the cartesian product of convex sets remains convex, it follows that $\mathcal{D}$ is a convex set.

Since the objective function in the problem is a quadratic form, with Q being a positive semidefinite matrix, as a result, the function is differentiable, and its gradient is well-defined across the entire domain. Additionally, since Q is positive semidefinite, the second derivative (the Hessian matrix) is symmetric and nonnegative.

For analyzing the convergence of the Frank-Wolfe algorithm, it is important to establish that the function is L -smooth. This property ensures that the gradient ∇f is Lipschitz continuous, meaning there exists a constant $L \geq 0$ such that [8]:

$$\|\nabla f(x) - \nabla f(y)\| \leq L\|x - y\|, \quad \forall x, y \in D.$$

This condition implies that the function does not exhibit sharp variations in its gradient, ensuring numerical stability and reliable convergence for optimization algorithms like Frank-Wolfe.

In the case of a twice-differentiable function, the Lipschitz continuity of the gradient is equivalent to requiring the operator norm of the Hessian $H(x)$ to be bounded above by L . Since the Hessian of the quadratic function (1) is constant and equal to $2Q$, the Lipschitz constant can be expressed in terms of the spectral properties of Q .

Specifically, the Lipschitz constant L is given by:

$$L = 2\|Q\| = 2\lambda_{\max}. \quad (5)$$

where $\lambda_{\max}$ is the largest eigenvalue of Q , and $\|Q\|$ denotes the spectral norm (operator norm) of Q .

This smoothness property is essential for the theoretical aspect on the convergence of the Frank-Wolfe algorithm, as it ensures that the objective function has bounded curvature.

3 Frank-Wolfe algorithm

The Frank-Wolfe algorithm, introduced by Marguerite Frank and Philip Wolfe in “An Algorithm for Quadratic Programming” (1956) [7], is a first-order optimization method

designed for convex constrained problems. Unlike traditional gradient methods, it avoids expensive projection steps by iteratively solving a linearized subproblem, making it efficient for structured domains like simplexes.

This property makes Frank-Wolfe widely applicable in context such as machine learning (e.g., SVMs), finance (portfolio allocation) and network optimization.

In this work, we take advantage of its strengths to solve a quadratic optimization problem over disjoint simplexes.

To achieve this, the function f is locally approximated using its first-order Taylor expansion:

$$L_x(y) = f(x) + (y - x)^T \nabla f(x), \quad \forall y \in D. \quad (6)$$

This approach allows us to replace the original nonlinear problem with a sequence of linearized subproblems that are easier to solve, particularly when the feasible domain has a structured representation, such as a set of disjoint simplices.

3.1 General Frank-Wolfe algorithm

The standard Frank-Wolfe algorithm operates as follows [2]:

- **Initialization:** Choose an initial point x_0 in the feasible set $\mathcal{D}$ and set iteration counter $t = 0$.

- **Step 1: Direction-finding subproblem**

Find s_t by solving the linear subproblem:

$$s_t = \arg \min_{s \in \mathcal{D}} \nabla f(x_t)^T s$$

This step minimizes the linear approximation of the objective function given by the first-order Taylor expansion of f around x_t , while remaining within the feasible set.

- **Step 2: Step size determination**

Select the step size γ_t using one of the following strategies:

- Fixed rule: $\gamma_t = \frac{2}{t+2}$.
- Exact line search: $\gamma_t = \arg \min_{\gamma \in [0,1]} f(x_t + \gamma(s_t - x_t))$

- **Step 3: Update**

Update the iterate:

$$x_{t+1} = x_t + \gamma_t(s_t - x_t)$$

Increment the iteration counter t and repeat from Step 1 until convergence criteria are met.

3.2 Frank-Wolfe algorithm over disjoint simplices

In our specific framework we can adapt the algorithm considering the structured of $\mathcal{D}$ consisting of disjoint simplices.

The Frank-Wolfe algorithm can be adapted to this problem as follows:

- **Initialization:** Choose an initial feasible point x_0 that satisfies the simplex constraints.
- **Gradient computation:** Compute the gradient $g = \nabla f(x_t)$.

- **Linearization step:** Solve the linear minimization problem for each simplex:

- For each $k \in K$, find:

$$s^k = \arg \min_{s_t \in \Delta^k} s_t^T g$$

- The optimal solution corresponds to selecting the index $i^* \in I^k$ where g_i is smallest:

$$s^k = e_{i^*}, \quad i^* = \arg \min_{i \in I^k} g_i$$

- **Search direction:** Define $d_t = s - x_t$
- **Step size selection:** Compute γ_t using a step size rule.
- **Update rule:** Set $x_{t+1} = x_t + \gamma_t d_t$.
- **Convergence check:** If $-d_t^T \nabla f(x_t) < \epsilon$ stop, where ϵ is a chosen predefined tolerance.

The key step in the Frank–Wolfe algorithm is solving the linearized subproblem over the feasible domain. For a set of disjoint simplices, this minimization simplifies due to the following properties used above:

Separability of the linear minimization subproblem

Since the feasible set is composed of disjoint simplices, the problem can be subdivided as follows:

$$s = \arg \min_{x \in \mathcal{D}} x^T g \Rightarrow s^k = \arg \min_{s \in \Delta_k} s^T g. \quad (7)$$

Thus, the optimization can be solved independently over each simplex Δ_k .

Minimization over a simplex

For each simplex Δ_k , the optimization reduces to:

$$\min_{s \in \Delta_k} s^T g = \min_{s \in \Delta_k} \sum_{i \in I^k} s_i g_i \quad (8)$$

Since s lies in a probability simplex (i.e., it sums to 1 and is nonnegative), the function $s^T g$ is minimized by choosing the unit vector corresponding to the smallest gradient component:

$$s^k = e_{i^*}, \quad i^* = \arg \min_{i \in I^k} g_i. \quad (9)$$

Thus, the optimal selection of s is given by:

$$s_i = \begin{cases} 1, & \text{if } i = i^*, \quad i^* = \arg \min_{i \in I^k} g_i, \\ 0, & \text{otherwise.} \end{cases} \quad (10)$$

In other words, we assign $s_{i^*} = 1$ to the index i^* that gets the minimum gradient value within I^k , while other components of s in the same partition are set to 0.

Proof: For each vector $s \in \mathcal{D}$ and the dot product with $g \in \mathbb{R}^n$ hold this inequality:

$$\sum_{i \in I^k} s_i g_i \geq \min\{g_i : i \in I^k\} \quad (11)$$

This is true because the left term is a convex combination and cannot be smaller than the smallest value in the partition.

So, it is equivalent to assign all the weight to those indexes where the gradient is minimized.

Descent property

The Frank–Wolfe search direction is given by:

$$d_t = s - x_t. \quad (12)$$

The function decrease is determined by the directional derivative:

$$\langle \nabla f(x_t), d_t \rangle = \langle g, s - x_t \rangle \leq 0. \quad (13)$$

Since s minimizes $g^T s$, it ensures the largest possible reduction in the function value at each iteration, leading to descent in the Frank–Wolfe algorithm.

Following Algorithm 1 showing the pseudocode concerning what has been discussed so far.

Algorithm 1 Frank–Wolfe Algorithm for Quadratic Optimization on Disjoint Simplices

```
1: procedure FW( $f, x, \text{partitions}, \epsilon$ )
2: repeat
3:   Compute the gradient:  $g = \nabla f(x_t)$ 
4:   Initialize a vector of 0's:  $s = \mathbf{0}$ 
5:   Solve the linearization problem for each simplex:
6:   for each partition in partitions do
7:      $i = \arg \min g[\text{partition}]$ 
8:   Assign all the weight where  $g$  is minimized:  $s[\text{partition}[i]] = 1$ 
9:   end for
10:  Construct search direction:  $d_t = s - x_t$ 
11:  Line search: Find  $\gamma_t = \arg \min_{\gamma \in [0,1]} f(x_t + \gamma d_t)$ 
12:  Update:  $x_{t+1} = x_t + \gamma_t d_t$ 
13: until  $-d_t \nabla f(x) < \epsilon$ 
end procedure
```

3.3 The Frank-Wolfe gap

The convergence analysis of the Frank-Wolfe (FW) algorithm heavily relies on gap measures that quantify optimality. Among the most common measures are the **primal gap**, **duality gap** and the **Frank-Wolfe gap** [4], which provide useful stopping criteria.

Intuitively, the gap measures the discrepancy between the current function value and the best possible improvement that could be achieved within the feasible domain. If this gap is small, it means that we are close to the optimal solution, as the best possible step results in a negligible improvement.

Primal gap

The **primal gap** quantifies the difference in function values between a given iterate x and the optimal solution x^* :

$$h(x) \triangleq f(x) - f(x^*) = \max_{y \in \mathcal{D}} f(x) - f(y). \quad (14)$$

Since the primal gap depends on the unknown optimal function value $f(x^*)$, it is generally not directly computable. To address this, alternative gap measures, such as the **Frank-Wolfe gap**, are used.

Frank-Wolfe gap

The **Frank-Wolfe gap** is an upper bound on the primal gap that does not require explicit knowledge of x^* . It is defined as:

$$g(x) \triangleq \max_{v \in \mathcal{D}} \langle \nabla f(x), x - v \rangle. \quad (15)$$

By convexity, it holds that:

$$0 \leq h(x) \leq \langle \nabla f(x), x - x^* \rangle \leq \max_{v \in \mathcal{D}} \langle \nabla f(x), x - v \rangle = -\langle \nabla f(x), d \rangle = g(x). \quad (16)$$

This bound demonstrates that the Frank-Wolfe gap can be used as a substitute for the primal gap, the duality gap and is particularly useful as a stopping criterion. Since computing $g(x)$ is a fundamental step in each FW iteration, it can be exploited efficiently.

Furthermore, if $g(x) = 0$, then x is a global minimum within $\mathcal{D}$. This property ensures that once the duality gap reaches zero, the iterates have converged to an optimal solution [4].

So, the Frank-Wolfe gap at iteration t is given by:

$$g(x_t) = \langle \nabla f(x_t), x_t - s_t \rangle = -\langle \nabla f(x_t), d_t \rangle, \quad (17)$$

where $s_t = \arg \min_{s \in \mathcal{D}} \langle \nabla f(x_t), s \rangle$ is the FW linear minimizer, and $d_t = s_t - x_t$ is the descent direction.

3.4 Step size selection

In the Frank-Wolfe algorithm, the step size plays an important role in determining how the iterates move toward the optimal solution. An appropriate choice of step size ensures faster convergence, stability, and numerical efficiency. One commonly used approach is the **exact step size**, which we will adopt for our algorithms. Another commonly used step size is the **diminishing step size**, given by:

$$\gamma_t = \frac{2}{t+2}. \quad (18)$$

While this step size ensures convergence, it suffers from slow convergence rates in practice. The step size decreases too quickly, leading to smaller updates and slow progress toward the optimum. Additionally, it does not adapt to the function's curvature, potentially causing inefficient iterations.

The **exact step size** is computed by solving the following optimization problem at each iteration:

$$\gamma_t = \arg \min_{\gamma \in [0,1]} f(x_t + \gamma d_t), \quad (19)$$

where:

- x_t is the current iterate,
- $d_t = s_t - x_t$ is the Frank-Wolfe search direction,
- $s_t = \arg \min_{s \in D} \langle \nabla f(x_t), s \rangle$ is the linear minimizer over the feasible set D ,
- $f(x)$ is the objective function.

This approach ensures that each step takes the exact minimizer along the current search direction, leading to faster convergence in many cases.

For the specific case where the objective function is given by (1), we derive the exact step size explicitly. Substituting $f(x)$ into the step size optimization problem:

$$f(x_t + \gamma d_t) = (x_t + \gamma d_t)^T Q (x_t + \gamma d_t) + q^T (x_t + \gamma d_t). \quad (20)$$

Expanding the expression:

$$f(x_t + \gamma d_t) = x_t^T Q x_t + 2\gamma x_t^T Q d_t + \gamma^2 d_t^T Q d_t + q^T x_t + \gamma q^T d_t. \quad (21)$$

Taking the derivative with respect to γ and setting it to zero:

$$2x_t^T Q d_t + 2\gamma d_t^T Q d_t + q^T d_t = 0. \quad (22)$$

Solving for γ_t :

$$\gamma_t = \frac{-(2x_t^T Q d_t + q^T d_t)}{2d_t^T Q d_t}. \quad (23)$$

Rearranging we get:

$$\gamma_t = \frac{-d_t(2Qx_t^T + q^T)}{2d_t^T Q d_t} = \frac{-d_t(\nabla f(x))}{2d_t^T Q d_t} = \frac{g(x)}{2d_t^T Q d_t}. \quad (24)$$

Since Q is positive semidefinite, it is possible that $d_t^T Q d_t = 0$, which would make the denominator zero. In such cases, the function is linear along d_t , meaning that the step size should be set to 1, as d_t is a descent direction, and the linear function is minimized by taking the step at the boundary.

3.5 Away-Step Frank-Wolfe

One of the main problems with the vanilla version of Frank-Wolfe's algorithm lies in the speed of convergence being limited by those vertices chosen in the initial phase, as they remain there in the convex combination of all future iterations bringing a zig-zagging behaviour.

The **Away-Step Frank-Wolfe (AFW)** algorithm addresses this issue by introducing away steps, which allow the removal of suboptimal vertices from the current iterate's convex decomposition. So, the idea is to move away from undesirable vertices in a given convex combination that could reduce convergence speed.

3.5.1 General Away-Step Frank-Wolfe

The standard Away-Step Frank-Wolfe algorithm follows the same general structure as the standard Frank-Wolfe method in Section 3.1 but includes an additional away-step direction. It operates as follows [9]:

- **Initialization:** Choose an initial point x_0 in the feasible set $\mathcal{D}$, set iteration counter $t = 0$ and define an active set $S_0 = \{x_0\}$. The active set S_t contains the vertices that form the convex combination of x_t , i.e.,

$$x_t = \sum_{v \in S_t} \alpha_v v, \quad \text{where} \quad \sum_{v \in S_t} \alpha_v = 1, \quad \alpha_v > 0 \quad \forall v \in S_t.$$

The active set evolves dynamically: new vertices may be added through Frank-Wolfe steps, while away steps can lead to the removal of vertices when $\gamma_t = \gamma_{\max}$ as it will be detailed later.

- **Step 1: Direction-finding subproblems**

Find the two directions:

- The Frank-Wolfe direction by solving the following linear problem as already seen:

$$s_t = \arg \min_{s \in \mathcal{D}} \langle \nabla f(x_t), s \rangle$$

and defining it as: $d_t^{FW} = s_t - x_t$.

- The Away direction is determined by selecting the most suboptimal vertex in terms of descent:

$$v_t = \arg \max_{v \in S_t} \langle \nabla f(x_t), v \rangle.$$

This means that v_t is the vertex in the active set S_t that contributes the least to the function decrease.

So, the Away direction is defined as: $d_t^A = x_t - v_t$.

- **Step 2: Direction selection**

The algorithm selects the direction, d_t^{FW} or d_t^A , that yields the highest reduction in the gap.

- **Step 3: Step size selection**

Compute the maximum allowable step size:

- If choosing d_t^{FW} , set $\gamma_{\max} = 1$.
- If choosing d_t^A , set: $\gamma_{\max} = \frac{\alpha_{v_t}}{1 - \alpha_{v_t}}$, where α_{v_t} is the weight of v_t in the convex representation of x_t .

The step size γ_t is then determined via line search:

$$\gamma_t = \arg \min_{\gamma \in [0, \gamma_{\max}]} f(x_t + \gamma d_t).$$

- **Step 4: Update**

Update the iterate:

$$x_{t+1} = x_t + \gamma_t d_t.$$

Weight update rules:

- Frank-Wolfe step (towards s_t):

$$\alpha_{s_t}^{t+1} = (1 - \gamma_t) \alpha_{s_t}^t + \gamma_t, \quad \alpha_v^{t+1} = (1 - \gamma_t) \alpha_v^t \quad \forall v \in S_t \setminus \{s_t\}.$$

If $s_t \notin S_t$, it is added to S_t with weight γ_t .

- Away step (moving away from v_t):

$$\alpha_{v_t}^{t+1} = (1 + \gamma_t) \alpha_{v_t}^t - \gamma_t, \quad \alpha_v^{t+1} = (1 - \gamma_t) \alpha_v^t \quad \forall v \in S_t \setminus \{v_t\}.$$

If $\gamma_t = \gamma_{\max}$, then $\alpha_{v_t} = 0$ and v_t is removed from S_t .

The computational cost of updating the weights is $O(|S_t|)$, where $|S_t|$ is the number of active vertices in the current iterate.

Drop Step Condition: A drop step occurs when a vertex is completely removed from the active set S_t . This happens in two cases:

- Frank-Wolfe step: If $\gamma_t = 1$, the previous iterates are completely replaced by s_t and all previous weights α_v are set to zero except for $\alpha_{s_t} = 1$, meaning that $S^{t+1} = \{s_t\}$.
- Away step: If $\gamma_t = \gamma_{\max}$, then $\alpha_{v_t} = 0$ and v_t is removed from S_t .

Thus, the drop step naturally arises in the away step when $\gamma_t = \gamma_{\max}$, as it removes a vertex from the active set. In contrast, in a Frank-Wolfe step with $\gamma_t = 1$, the active set is entirely replaced, which can be seen as a full reset rather than a vertex selective removal.

- **Step 5: Convergence check**

If $-d_t \nabla f(x_t) < \epsilon$ stop, where ϵ is a chosen predefined tolerance.

3.5.2 Away-Step Frank–Wolfe algorithm over disjoint simplices

In our specific framework we can adapt again the algorithm considering the structure of $\mathcal{D}$ consisting of disjoint simplices.

In general, the away direction is determined by maximizing the dot product with the gradient over the entire active set. In our case, the problem naturally decomposes into independent simplices, allowing us to solve the subproblems separately for each simplex.

To reflect this, we introduce a modification where the maximization is done independently within each simplex. Specifically, instead of computing the away vertex from a global active set, we define a local one $S^k(x)$ restricted to each simplex Δ^k .

The away vertex is then selected as the one maximizing the gradient within each individual simplex, as we will see later.

A global notion of **active indices** can be denoted as: $S(x) = \{i : x_i > 0\}$. In particular, we can denote $S^k(x)$ as the set of active indices for each partition I^k , i.e.:

$$S^k(x) = \{i \in I^k : x_i > 0\}.$$

In particular, the relationship between the active set defined in the previous section and the active indices can be formulated as follows. Given that each active index $i \in S^k(x)$ has a unique representation in the basis e_i , the iterate x_t can be expressed as a convex combination of these vertices:

$$x_t = \sum_{k \in K} \sum_{i \in S^k(x_t)} \alpha_i e_i, \quad (25)$$

where:

- $S^k(x_t)$ represents the set of active indices within the k -th simplex,
- e_i is the canonical vertex corresponding to index i ,
- α_i are the convex coefficients satisfying $\sum_{i \in S^k(x_t)} \alpha_i = 1$ for each k .

A key reason for adopting an index-based formulation is the nature of the feasible domain $\mathcal{D}$, which consists of disjoint simplices. This formulation maintains a relation with the standard Away-Step Frank-Wolfe algorithm while considering the separability of the simplices, ensuring that each update step operates independently within each partition I^k .

We add a further direction, called **away direction**, pointing away from the bad vertices defined as:

$$d_t^A = x_t - v_t. \quad (26)$$

This direction, d_t^A , shifts the weight away from the worst vertex to the other vertices used to represent the iterate x_t . Here the worst vertex (among those having positive weight in current iterate, i.e. belonging to $S^k(x)$) is the one with the greatest scalar product with the gradient, or, equivalently, the one that maximizes the linear approximation of f given by $\nabla f(x_t)$ [1].

Given the nature of the problem, the strategy adopted to find the new direction is the same as that used in 3.2: the only difference lies in finding the indices of the active sets and considering the index where the gradient is maximum for each disjoint partition (Lines 15–18).

In particular, the vector v_t in 26 is a sparse vector that has:

- 1 at the position corresponding to the index with the maximum gradient value within each partition I^k considering only the indexes $S^k(x)$, i.e. the active vertices. So, we define:

$$v^k = e_{i^*}, \quad i^* = \arg \max_{i \in S^k(x_t)} g_i,$$

where v^k is the partial solution for the k -th partition.

- 0 in all other positions.

In a more compact way:

$$(v_t)_i = \begin{cases} 1, & \text{if } i = i_k^* \text{ for some } k \in \{1, \dots, K\}, \\ 0, & \text{otherwise.} \end{cases}$$

The next step is to decide which direction to choose between the two available. A natural approach is to evaluate which of the two is better on the basis of the improvement made in terms of the gap. So, if the improvement is greater for the FW direction, choose the FW direction and set the maximum step as $\gamma_{max} = 1$ (Lines 22–24); vice versa, choose the direction provided by the Away-Step and set the maximum step as defined in line 18 so that the total weight within each simplex remains within the constraints. In addition, we adopt a conservative approach by taking the minimum allowable step size across all simplices. This guarantees that the updating remains within the feasible region while making the maximal admissible progress.

The subsequent lines of the algorithm are almost identical to Algorithm 1 with the only exception that the line search is limited by γ_{max} along the direction that was previously chosen to ensure feasibility when performing an away step.

Algorithm 2 Away-Step Frank–Wolfe Algorithm for Quadratic Optimization on Disjoint Simplices

```

1: procedure FW( $f$ ,  $x$ , partitions,  $\epsilon$ )
2: repeat
3:   Compute the gradient:  $g = \nabla f(x_t)$ 

4:   Initialize a vector of 0's:  $s = \mathbf{0}$ 
5:   Solve the min linearization problem for each simplex:
6:   for each partition in partitions do
7:      $i = \arg \min g[\text{partition}]$ 
8:     Assign all the weight where  $g$  is minimized:  $s[\text{partition}[i]] = 1$ 
9:   end for
10:  Construct FW search direction:  $d_t^{FW} = s - x_t$ 

11:  Initialize a vector of 0's:  $v = \mathbf{0}$ 
12:  Find active set:  $active = x > 0$ 
13:   $\gamma_{max} = inf$ 
14:  Solve the max linearization problem for each simplex:
15:  for each partition in partitions do
16:    Select indexes:  $S^k = \text{partition}[active]$ 
17:     $i = \arg \max g[S^k]$ 
18:    Assign all the weight where  $g$  is maximized:  $v[S^k[i]] = 1$ 
19:     $\gamma_{max} = \min\{\gamma_{max}, \frac{x[S^k[i]]}{1-x[S^k[i]]}\}$ 
20:  end for
21:  Construct AFW search direction:  $d_t^A = x_t - v$ 

22:  if  $g \cdot d_t^{FW} \geq g \cdot d_t^A$  then
23:     $d = d_t^{FW}$ 
24:     $\gamma_{max} = 1$ 
25:  else
26:     $d = d_t^A$ 
27:  end if

28:  Line search: Find  $\gamma_t = \arg \min_{\gamma \in [0, \gamma_{max}]} f(x_t + \gamma d_t)$ 
29:  Update:  $x_{t+1} = x_t + \gamma_t d_t$ 
30: until  $-d_t \nabla f(x) < \epsilon$ 
end procedure

```

4 Convergence analysis

4.1 Convergence analysis for Vanilla Frank-Wolfe

The classical convergence result for the vanilla Frank-Wolfe algorithm states that:

Theorem 1 (Primal convergence of the Frank-Wolfe algorithm [11]). *Let f be an L -smooth convex function and let P be a compact convex set of diameter D . Then the following holds:*

$$f(x_t) - f(x^*) \leq \frac{2LD^2}{t+2}, \quad (27)$$

and hence for any accuracy $\epsilon > 0$ we have:

$$f(x_t) - f(x^*) \leq \epsilon \quad \text{for all } t \geq \frac{2LD^2}{\epsilon},$$

where:

- L is the Lipschitz constant, defined as (5).
- D is the diameter of D , i.e., $D = \max_{x,y \in \mathcal{D}} \|x - y\|$.
- $f(x^*)$ is the optimal value of the problem.
- t is the iteration count.

This bound demonstrates that the Frank-Wolfe method exhibits a sublinear rate of $O(1/k)$ worst-case convergence rate in terms of function value.

An alternative way to measure convergence is through the Frank-Wolfe gap, defined as (15).

As already mentioned, this quantity is used as an upper bound on the primal gap and provides a stopping criterion that does not require prior knowledge of the diameter D or the Lipschitz constant L . Specifically, we can terminate the algorithm once this gap falls below a target accuracy ϵ .

Theorem 2 (Frank-Wolfe gap convergence of the Frank-Wolfe algorithm.). *Let f be an L -smooth convex function and let P be a compact convex set of diameter D . Then the running minimum of the Frank-Wolfe gaps up to iteration t satisfies:*

$$\min_{0 \leq \tau \leq t} \max_{v_\tau \in P} \langle \nabla f(x_\tau), x_\tau - v_\tau \rangle \leq \frac{6.75LD^2}{t+2}. \quad (28)$$

The key reason for the sublinear rate are related from the structure of the update step in Frank-Wolfe. Unlike projection-based gradient methods, Frank-Wolfe moves along a linear segment from the current iterate towards an extreme point of $\mathcal{D}$, leading to the following key limitations. First, the limited descent per iteration arises because the algorithm chooses a direction that minimizes the linearized objective function, meaning the descent is not always in the steepest direction for $f(x)$. Second, the method's speed depends on the curvature constant L , which determines how quickly the gradients change. Higher curvature slows convergence as the linearization may become a poor approximation of $f(x)$.

However, under certain function assumptions FW's algorithm shows linear convergence.

In particular, a differentiable function f is said to be c -gradient dominated for a constant $c > 0$ if for all x and y in its domain, the following inequality holds:

$$\frac{f(x) - f(y)}{c^2} \leq \|\nabla f(x)\|_*^2. \quad (29)$$

This property works as a generalization of strong convexity and ensures that the function decreases sufficiently fast in the direction of its negative gradient. Intuitively, a function that is c -gradient dominated exhibits a contraction property that allows the Frank-Wolfe algorithm to achieve linear convergence.

Additionally, a function f is μ -strongly convex if there exists a constant $\mu > 0$ such that for all x, y in its domain [3]:

$$f(y) \geq f(x) + \nabla f(x)^T(y - x) + \frac{\mu}{2}\|y - x\|^2. \quad (30)$$

This property ensures not only the uniqueness of the optimal solution but also a quadratic lower bound on the function's curvature. Strongly convex functions are automatically c -gradient dominated with $c = \frac{1}{\sqrt{2\mu}}$ [4], meaning that the convergence properties of the Frank-Wolfe algorithm under strong convexity follow directly from the more general c -gradient dominance assumption.

Theorem 3 (Linear Convergence of the Frank-Wolfe Algorithm [4]). *Let $\mathcal{X}$ be a compact convex set with diameter D and f an L -smooth and c -gradient dominated function. Assume further there exists a minimal place $x^* \in \text{Int}(\mathcal{X})$ of f in the interior of $\mathcal{X}$, i.e., there exists an $r > 0$ with $B(x^*, r) \subset \mathcal{X}$. Then the (vanilla) Frank-Wolfe algorithm's iterates x_t satisfy:*

$$f(x_t) - f(x^*) \leq \left(1 - \frac{r^2}{2Lc^2D^2}\right)^{t-1} \frac{LD^2}{2}, \quad (31)$$

for all $t \geq 1$. Equivalently, the primal gap is at most $\varepsilon > 0$ after the following number of linear optimizations and gradient computations:

$$1 + \frac{2Lc^2D^2}{r^2} \ln \frac{LD^2}{2\varepsilon}. \quad (32)$$

In particular, if f is μ -strongly convex (and hence $c = \frac{1}{\sqrt{2\mu}}$ -gradient dominated), then for a primal gap at most $\varepsilon > 0$, at most the following number of linear optimizations and gradient computations are needed:

$$1 + \frac{LD^2}{\mu r^2} \ln \frac{LD^2}{2\varepsilon}. \quad (33)$$

4.2 Convergence analysis for Away-Step Frank-Wolfe

Before analyzing the convergence for the Away-Step Frank-Wolfe (AFW) algorithm, it is important to highlight the problems encountered in previous convergence analyses.

Traditional approaches demonstrate limitations when the optimal solution x^* was close to a boundary without lying on it, resulting in slower convergence rates [10].

To deal with this, Lacoste-Julien and Jaggi (2015) [9] introduced the **pyramidal width** δ , a "geometric quantity" that refines existing metrics like the vertex-facet distance¹. This

¹The vertex-facet distance is defined as:

$$\min_{F \in \text{facets}(\text{conv}(A))} \text{dist}(\text{affine}(F), \text{vertices}(\text{conv}(A \setminus F))).$$

It measures the distance between the affine hull of a facet and the vertices of the convex hull obtained by removing that facet. This is more local than the pyramidal width, which is a global measure over all faces of the polytope.

constant ensures that the convergence rate of AFW remains stable and independent of the position of x^* within the feasible region.

Instead of the original definition, we recall an equivalent characterization [4], which offers a direct proof as follows:

Theorem 4 (Pyramidal width as facial distance [4]). *For any polytope P , its pyramidal width δ is the minimum distance of any of its faces F from the convex hull of all vertices of P not lying on F , i.e.,*

$$\delta = \min_{F \text{ face of } P} \text{dist}(F, \text{conv}(\text{Vert}(P) \setminus F)), \quad (34)$$

where we restrict to proper faces F , i.e., $F \notin \{\emptyset, P\}$; alternatively, one may treat the distance between the empty set and P as infinite.

The introduction of δ provides a tool to achieve a more stable solution.

To further investigate the convergence analysis, it is important to derive an upper bound on the primal gap, which is useful to track the progress at each iteration.

Theorem 5 (Geometric Strong Convexity [4]). *Let P be a polytope with pyramidal width $\delta > 0$, and let f be a μ -strongly convex function. Then, we can define at the t -th iteration the Frank–Wolfe vertex and the away vertex, respectively, as:*

$$v_t^{FW} = \arg \min_{v \in P} \langle \nabla f(x_t), v \rangle, \quad v_t^A = \arg \max_{v \in S} \langle \nabla f(x_t), v \rangle,$$

where $S \subseteq \text{Vert}(P)$ such that $x_t \in \text{conv}(S)$. Considering $x = x_t$, we obtain:

$$f(x_t) - f(x^*) \leq \frac{\langle \nabla f(x_t), v_t^A - v_t^{FW} \rangle^2}{2\mu\delta^2}.$$

To conclude the analysis, we examine the condition under which the AFW algorithm achieves linear convergence and the circumstances that enable this behavior.

In particular, when $\gamma_t = \gamma_{t,\max}$ during an away step, the active set is reduced by removing the away vertex v_t^A , a process called **drop step**. This operation can occur in at most half of the iterations, as a vertex can only be removed if it was previously added in an earlier iteration. A demonstration can be found in the following theorem.

Theorem 6 (Linear convergence of the AFW algorithm [4]). *Let $P \subset \mathbb{R}^n$ be a polytope and f be an L -smooth and μ -strongly convex function over P . The convergence rate of the AFW with objective f is linear. For all $t \geq 1$*

$$f(x_t) - f(x^*) \leq \left(1 - \frac{\mu}{4L} \frac{\delta^2}{D^2}\right)^{\lceil (t-1)/2 \rceil} \frac{LD^2}{2},$$

where D and δ are respectively the diameter and the pyramidal width of the polytope P .

If the objective function is smooth but not necessarily convex, the convergence rate of the AFW algorithm matches that of the vanilla FW algorithm (see Equation 27).

This concludes the first part of our convergence analysis, providing a baseline to compare the performance of these two algorithms.

We have reported the convergence analysis based on the standard AFW, as its theoretical properties extend naturally to our disjoint-simplices problem. The index-based formulation preserves the key properties of the classical algorithm (see the relation in 25) while exploiting the separability of the simplices, ensuring that each update step operates independently within each partition I^k .

This decomposition does not modify the fundamental logic of the algorithm but instead adapts it to better fit the problem's natural partitioning. Thus, the optimization can be solved independently on each simplex and then combined, aligning with the standard AFW update mechanism ensuring that the global update step remains equivalent to the standard AFW one. Consequently, the theoretical convergence results remain applicable under similar conditions, and the experimental evaluation will further confirm whether our specialized implementation aligns with the expected performance of standard AFW.

4.3 Constants estimation

The Lipschitz constant L has already been estimated in Section 2, where we derived it as $L = 2\lambda_{\max}$, with $\lambda_{\max}$ being the largest eigenvalue of the matrix Q .

Regarding the remaining constants: the diameter D , the strong convexity parameter μ , and the pyramidal width δ , it is also appropriate to estimate them, as they are fundamental for obtaining a precise ex-ante estimate of the convergence speed.

Diameter

The diameter of a single standard simplex Δ_k is known to be [4]:

$$D_k = \sqrt{2}.$$

Since there are K disjoint simplices in the domain $\mathcal{D}$ the overall diameter is given by:

$$D \leq \sqrt{2K}.$$

This estimate provides an upper bound on the maximum distance within the feasible region.

μ -constant

To estimate the parameter μ , we exploit the relationship between strong convexity and the eigenvalues of the Hessian matrix Q , which in our case is a positive semidefinite matrix.

Due to the strongly convex property, it follows that all the eigenvalues of the Hessian matrix $\nabla^2 f(x)$ are greater than or equal to μ , ensuring a lower bound on the curvature of f . Since in our case $\nabla^2 f(x) = 2Q$, we obtain [3]:

$$2Q \succeq \mu I \succeq 0 \quad \Rightarrow \quad \mu \leq 2\lambda_{\min}(Q).$$

If Q is positive definite, we can say that f is strongly convex with:

$$\mu \approx 2\lambda_{\min}(Q),$$

where $\lambda_{\min}(Q)$ is the smallest positive eigenvalue of the matrix Q .

If instead Q is only positive semidefinite, we may have $\lambda_{\min} = 0$, which implies that f is not strictly convex along certain directions, slowing down the algorithm's convergence and $\mu = 0$, meaning that the function does not satisfy the strong convexity condition.

Pyramidal width

If we consider a probability simplex with d vertices, the pyramidal width is simply the same as its width, which is given by [9]:

$$\delta = \begin{cases} \frac{2}{\sqrt{d}}, & \text{if } d \text{ is even,} \\ \frac{2}{\sqrt{d-\frac{1}{d}}}, & \text{if } d \text{ is odd.} \end{cases} \quad (35)$$

Considering our specific domain composed of disjoint simplices we can use a conservative lower bound to estimate δ taking the minimum pyramidal width along all the partitions:

$$\delta = \min_k \delta_k,$$

where k indicates the partitions and δ_k is calculated using 35 . This estimation ensures that our estimate still valid for the worst-case simplex.

5 Experiments

5.1 Instance generation and initial point selection

In this section, we describe the process used to generate synthetic data for our experiments. The goal of data generation is to create instances that respect the mathematical properties of the problem while allowing control over key parameters. This enables us to empirically observe how different configurations impact algorithms performance.

Data generation

The first step in data generation is constructing the quadratic function $f(x)$, which consists of a positive semidefinite (PSD) matrix Q and a linear term q . The properties of Q can significantly impact the optimization problem, affecting the problem conditioning and potentially slowing the algorithm convergence.

Algorithm 3 contains the related pseudocode, in particular analyzing the input parameters we have:

- n : the problem dimension, determining the size of Q and q .
- K : the number of partitions.
- $\rho(Q)$: the spectral radius (largest eigenvalue) of matrix Q , influencing its conditioning.
- λ_{min} : the smallest positive eigenvalue of matrix Q .
- $density$: the fraction of nonzero entries in Q , controlling matrix sparsity.
- $q_generation$: the strategy adopted for generating q .
- $violated_constraint$: the parameter to select the desired violation ('sum_simplex', 'non_negative', 'all').
- $dim_ker(Q)$: the dimension of the null space of Q .
- $seed$: set a random seed for reproducibility.

The pseudocode first performs an input validity check, ensuring that parameters are within their acceptable ranges (e.g., n must be strictly positive).

Then, from line 9 to line 16 takes place the matrix generation part. In particular, Q is generated through a structured eigenvalue decomposition approach: all the eigenvalues are initialized to 0, ensuring that the matrix will be PSD. Then, using *linspace* from the library NumPy [6] it's possible to assign the first $n - dim_ker(Q)$ eigenvalues in an evenly spaced manner between $\rho(Q)$ and λ_{min} , ensuring numerical stability and avoiding extreme conditioning while the remaining $dim_ker(Q)$ eigenvalues still equal to zero, enforcing a controlled null space. Following a random matrix is drawn from a normal distribution and subjected to QR decomposition to obtain an orthonormal basis U and using the spectral decomposition of a symmetric matrix we obtain the matrix Q .

Now, we can observe three possible scenarios:

- $dim_ker(Q) = n$: In this case, the matrix Q is the zero matrix, meaning all its eigenvalues are zero. Hence the problem is linear and it reduces to $f(x) = \langle q, x \rangle$
- $dim_ker(Q) = 0$: all eigenvalues of Q are strictly positive meaning that there is unique solution and $Q \succ 0$.

- $0 < \dim_{ker}(Q) < n$: this is the intermediate case, meaning that Q has both zero and positive eigenvalues. This implies that the function has flat directions along $ker(Q)$, meaning $f(x)$ behaves linearly in those directions.

Once Q is generated, an optional sparsification step randomly sets a proportion of its entries to zero to control density. The matrix is then re-symmetrized to ensure it remains valid.

In order to create unbiased and random partitioning a random permutation of the indices $\{0, \dots, n-1\}$ is performed (line 17). The shuffled indices obtained are then split into K disjoint subsets, ensuring each partition contains approximately n/K elements.

The final step is generating the vector q . The generation strategy is controlled by the parameter `q_generation`, which determines the position of the unconstrained minimizer. Three cases are considered:

- `random`: q is generated as a normalized vector sampled from a standard Gaussian distribution. This produce a general configuration with no special structure.
- `internal`: In this case, q is defined as $q = -2Qx$, where x is a feasible point. This ensures that the unconstrained minimizer lies inside the admissible region.
- `external`: Also in this case $q = -2Qx$, but x is built to violate one or more constraints. The type of violation is specified by the parameter `violated_constraint`, which can be:
 - `sum_simplex`: alters the sum of each simplex.
 - `non_negative`: introduces negative components.
 - `all`: combines both effects.

These configurations will support a controlled comparison of convergence behavior between different scenarios.

Algorithm 3 Data Generation

```
1: procedure GENERATE_DATA( $n, K, \rho(Q), \lambda_{\min}, \text{density}, \dim_{\ker}(Q), q\_generation,$   
    $\text{violated\_constraint}, \text{seed}$ )  
  
2: Input:  $n, K, \rho(Q), \lambda_{\min}, \text{density}, \dim_{\ker}(Q), q\_generation, \text{violated\_constraint}, \text{seed}$   
3: Output:  $Q \in \mathbb{R}^{n \times n}$  (PSD matrix),  $q \in \mathbb{R}^n$ ,  $\text{partitions}$   
  
4: Check input validity:  
5: if  $n \leq 0$  or  $\rho(Q) \leq 0$  or  $\lambda_{\min} \notin (0, \rho(Q)]$  or  $\text{density} \notin [0, 1]$  or  $\dim_{\ker}(Q) \notin [0, n]$  or  
    $K \notin [1, n]$  or  $\text{seed} \notin \mathbb{Z}$  then  
6:   Raise error  $\triangleright$  Invalid parameters  
7: end if  
  
8: Set random seed  
  
9: Initialize  $\text{eigenvalues} = \mathbf{0}_n$   
    $\triangleright$  Generates  $n - \dim_{\ker}(Q)$  eigenvalues evenly spaced from  $\rho(Q)$  to  $\lambda_{\min}$ .  
10:  $\text{eigenvalues}[0 : n - \dim_{\ker}(Q)] \leftarrow \text{linspace}(\rho(Q), \lambda_{\min}, n - \dim_{\ker}(Q))$   
  
11:  $U \leftarrow \text{QR}(\text{randn}(n, n))$   $\triangleright$  Random orthonormal matrix  
12:  $Q \leftarrow U \cdot \text{diag}(\text{eigenvalues}) \cdot U^T$   
  
13: if  $\text{density} < 1.0$  then  
14:   Mask  $Q$  randomly  $\triangleright$  Impose sparsity  
15:    $Q \leftarrow (Q + Q^T)/2$   $\triangleright$  Ensure symmetry  
16: end if  
  
    $\triangleright$  Generates a random permutation of integers  $[0, n - 1]$  to ensure unbiased partitioning.  
17: Shuffle indices:  $\text{indices} \leftarrow \text{randperm}(n)$   
18: Partition indices into  $K$  groups:  $\text{partitions} \leftarrow \text{split}(\text{indices}, K)$   
  
    $\triangleright$  Generate feasible starting point  $x$  and partition indices  
19:  $x \leftarrow \text{START\_POINT}(n, \text{partitions}, \text{seed})$   
  
20: if  $q\_generation = \text{'random'}$  then  
21:    $q \leftarrow \text{randn}(n)$   
22:   Normalize  $q$   
  
23: else if  $q\_generation = \text{'internal'}$  then  
24:   Initialize  $x \leftarrow \mathbf{0}_n$   
25:   Set  $x$  with random values normalized within each partition  
  
26: else if  $q\_generation = \text{'external'}$  then  
27:   if  $\text{violated\_constraint} = \text{'sum\_simplex'}$  then  
28:     Multiply each simplex by random factor  $\in [0.5, 1.5]$   
29:   else if  $\text{violated\_constraint} = \text{'non\_negative'}$  then  
30:     For each simplex:  
31:       Select 10% of entries to become negative  
32:       Rebalance positive entries to sum to 1  
33:   else if  $\text{violated\_constraint} = \text{'all'}$  then  
34:     Combine both violation constraints  
35:   end if  
  
36:    $q \leftarrow -2Qx$   
37: end if  
  
38: return  $Q, q, \text{partitions}$   
39: end procedure
```

Starting point generation

We describe the procedure for generating a feasible starting point x_0 . Since our problem is a quadratic program over disjoint simplices, the initialization of x_0 must respect the constraint structure of $\mathcal{D}$ defined in 2.

Algorithm 4 constructs a valid feasible point taking as input:

- n : the dimension of x_0 .
- `partitions`: list of index arrays defining the simplices.

The pseudocode first performs an input validity check, ensuring that the total number of variables n is strictly positive.

Then, for each partition I^k , the algorithm uniformly distributes the weight across the corresponding indices. Specifically, each element within the partition is assigned an equal value of $\frac{1}{|I^k|}$, ensuring that they satisfy the simplex constraint.

As final output, the function return the feasible initial point x_0 .

Algorithm 4 Starting Point Generation

```
1: procedure START_POINT( $n$ , partitions)  
  
2: Input:  $n, partitions$   
3: Output:  $x_0$  (feasible initial point)  
  
4: Check input validity:  
5: if  $n \leq 0$  then  
6:   Raise error  $\triangleright$  Invalid parameters  
7: end if  
  
8: Initialize  $x_0 = \mathbf{0}_n$   
9: for each partition  $idx$  in partitions do  
10:   Compute uniform value:  $simplex\_vals \leftarrow \frac{1.0}{len(idx)}$   
11:   Assign to  $x_0$ :  $x_0[idx] \leftarrow simplex\_vals$   
12: end for  
  
13: return  $x_0$   
14: end procedure
```

5.2 q - generation strategy: results comparison

The first step of our convergence analysis focused on comparing the behavior of Frank-Wolfe (FW) and Away-Step Frank-Wolfe (AFW) across different strategies for generating q .

Specifically, the configuration used to generate all instances in this experiment was fixed as follows:

- Problem dimension: $n = 100$
- Number of simplices: $K = 20$
- Spectral radius of $Q = 5$
- Minimum eigenvalue of $Q = 1$
- Density = 1.0 (fully dense matrix)
- Kernel dimension = 0

The stopping criterion was based on a F-W gap tolerance of $\epsilon = 10^{-6}$, with a maximum number of iterations set to 50000.

Table 1 reports the results for each q -generation configuration. In particular, it shows the number of iterations, execution time, F-W gap and the relative gap value for both FW and AFW methods. The relative gap is calculated, using the optimal $f(x^*)$ obtained by the solver gurobi.

The results confirm a significant performance gap between FW and AFW in several settings. In the *random* and *internal* cases, FW tends to hit the maximum number of iteration without reaching the target precision, whereas AFW achieves convergence in a fraction of the time. In contrast, the *external* scenarios, highlight the robustness of AFW in handling challenging optimum positions. The combination of away steps and exact line search leads to improved convergence both in terms of speed and F-W gap.

	Iterations		Time [s]		F-W Gap		Relative Gap	
	FW	AFW	FW	AFW	FW	AFW	FW	AFW
Random	max	590	4.1717	0.01084	1.53e-03	9.94e-07	6.84e-05	-1.20e-09
Internal	max	230	4.1908	0.0374	8.07e-04	9.74e-07	1.25e-05	-1.72e-09
Ext sum_simplex	3469	317	0.2832	0.0709	9.88e-07	9.72e-07	7.84e-14	7.02e-15
Ext non_negative	max	298	4.2102	0.0499	1.86e-03	9.34e-07	2.84e-05	-3.01e-09
Ext all	max	542	4.1754	0.1008	1.91e-03	9.70e-07	1.00e-04	-1.01e-11

Table 1: Comparison of FW/AFW across different configurations

As shown in Figure 1, the convergence behavior across different q -generation strategies varies considerably between FW and AFW.

In particular, AFW consistently shows faster convergence, with steeper slopes and earlier drops in the F-W gap, as expected by the theoretical analysis.

For the **external_sum_simplex** violation, the convergence of FW appears to be as fast as AFW before slowing down, while AFW converges more rapidly and consistently. The **external_non_negative** and **external_all** settings further confirm AFW's adaptability: despite the unfavorable optimum position, AFW achieves convergence in far fewer iterations.

The **random** and **internal** configurations highlight one of the limitations of vanilla FW: the algorithm tends to reach a plateau, showing only sublinear convergence. This is explained by the presence of the so-called "zig-zagging behavior", discussed in Section 3.5. This is visible in the flat regions or small oscillations in the FW curves. On the other hand, AFW is able to perform away steps to escape such regions, leading to faster descent.

To conclude this first part of the analysis, before moving on to other comparisons, these experimental results visually confirm the theoretical predictions discussed.

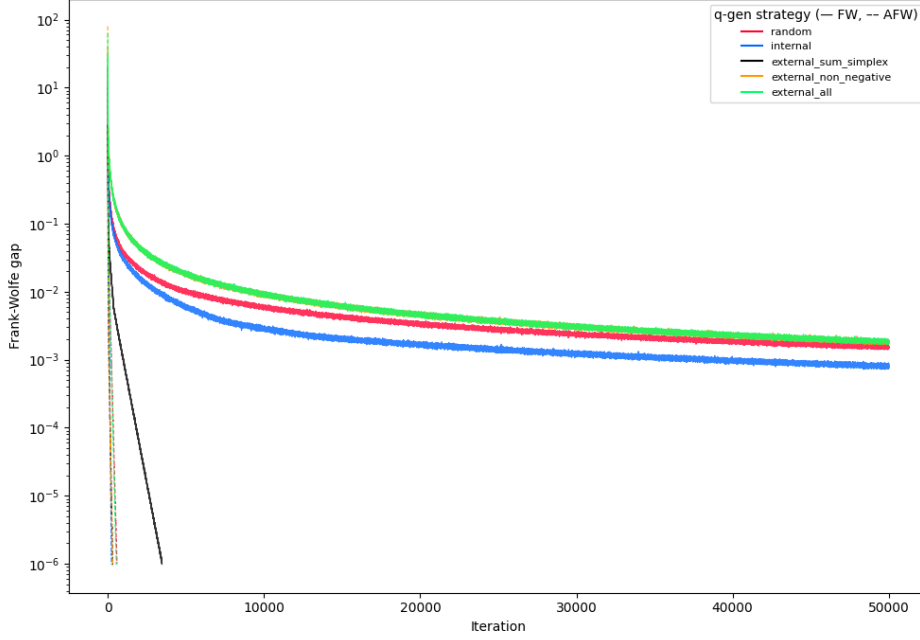


Figure 1: Convergence curves for q-generation strategies

5.3 Impact of the dimensionality

In this section, we investigate how the performance of FW and AFW algorithms is affected by the dimensionality of the problem n and the number of disjoint simplices K . The goal is to evaluate how scalability impacts the results, in particular in terms of convergence speed and computational time.

To isolate the effect of dimensionality, we fixed the following parameters:

- Spectral radius of $Q = 5$
- Minimum eigenvalue of $Q = 1$
- Density = 1.0 (fully dense matrix)
- Kernel dimension = 0

We then explored different combinations of n and K as follows:

- $K = 2$, with $n = 10, 100, 1000$
- $K = 20$, with $n = 100, 1000$
- $K = 200$, with $n = 1000$

This setup allows us to observe the behavior of the algorithms under increasing problem sizes, both in terms of variable count and partition number.

The stopping criterion was based on a F-W gap tolerance of $\epsilon = 10^{-6}$, with a maximum number of iterations set to 50000.

Table 2 reports the performance metrics for FW and AFW under the specified settings, across all q-generation strategies. As in the previous section, this strategies variation has been preserved throughout this experiment and will continue to be maintained across all subsequent experiments.

From a general perspective, the results show a consistently superior performance of AFW in terms of number of iterations, execution time, and gaps.

As expected, increasing the problem dimension n and/or the number of partitions K leads to a notable rise in computational time for both methods.

In particular, FW reaches the maximum iteration limit in almost all settings, even for relatively small problems (e.g., $n = 10$). As the problem size increases, we can observe a corresponding growth in computational time. By contrast, AFW also starts to suffer from increased dimensionality, but it consistently reaches the target F-W gap in a number of iterations well below the maximum threshold. Consequently, as K increases, the execution time also grows, since more "work" is required to optimize each individual partition. Moreover, in terms of relative gap, AFW provides a better approximation of the optimal solution across all tested configurations.

Figure 2 provides a visual comparison of the convergence behavior of FW and AFW for the `sum_simplex` violation under two representative dimensionality settings.

Subfigure 2a shows the case with $N = 100$ and $K = 2$, one of the few configurations where FW successfully converges within the iteration limit. Despite FW reaching the target F-W gap, its convergence curve is visibly flatter and slower compared to AFW, which rapidly decreases the gap in the early iterations.

Subfigure 2b shows the scenario with $N = 1000$ and $K = 200$, where the problem size is significantly larger. In this case, FW fails to reach the desired precision tolerance, while AFW still converges within the iteration limit. However, the curve is noticeably less steep than in smaller instances, reflecting the increased computational effort required as both n and K grow.

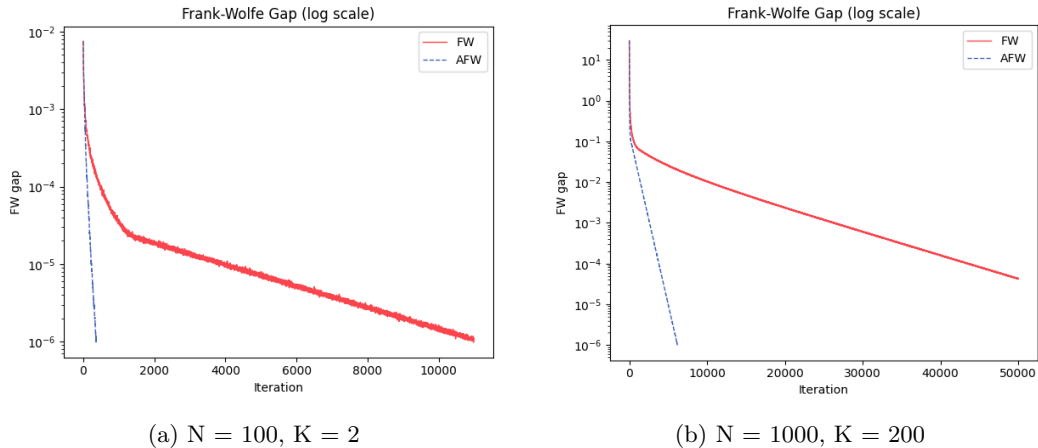


Figure 2: Convergence curves for sum simplex violation

	Iterations		Time [s]		F-W Gap		Relative Gap	
	FW	AFW	FW	AFW	FW	AFW	FW	AFW
N = 10, K = 2								
Random	9092	58	0.2339	0.0021	9.93e-07	8.30e-07	1.55e-10	5.56e-13
Internal	440	45	0.0112	0.0018	9.85e-07	9.25e-07	1.34e-12	9.82e-14
Ext sum_simplex	361	45	0.0092	0.0017	9.73e-07	9.36e-07	1.24e-12	9.54e-14
Ext non_negative	max	36	1.2868	0.0014	1.44e-04	7.17e-07	1.55e-05	-7.25e-11
Ext all	max	36	1.2714	0.0014	1.46e-04	8.02e-07	4.17e-05	-2.21e-09
N = 100, K = 2								
Random	max	366	3.2036	0.0627	2.43e-04	9.94e-07	4.06e-03	-5.15e-08
Internal	max	594	3.1769	0.0738	8.76e-05	9.79e-07	1.17e-04	-5.69e-09
Ext sum_simplex	10982	363	0.6746	0.0635	9.80e-07	9.90e-07	6.06e-10	1.01e-11
Ext non_negative	max	214	3.2141	0.0277	2.34e-04	9.80e-07	7.55e-05	-8.22e-09
Ext all	max	201	3.4305	0.0271	2.28e-04	8.92e-07	9.16e-05	-7.43e-11
N = 1000, K = 2								
Random	max	1731	51.9127	2.4740	2.42e-04	9.94e-07	2.73e-03	-7.20e-09
Internal	max	4175	52.0013	5.6494	7.59e-05	9.94e-07	7.29e-04	-5.49e-08
Ext sum_simplex	max	3302	52.0347	4.8301	4.08e-06	9.99e-07	1.32e-06	9.79e-10
Ext non_negative	max	668	54.3544	0.9060	2.21e-04	9.38e-07	4.32e-05	-5.76e-11
Ext all	max	784	55.5063	1.0626	2.31e-04	9.40e-07	9.55e-05	-1.16e-09
N = 1000, K = 20								
Random	max	4823	58.5773	7.3900	1.68e-03	9.93e-07	7.99e-04	-1.77e-11
Internal	max	4229	57.4464	6.7666	7.60e-04	9.92e-07	7.61e-05	-4.73e-11
Ext sum_simplex	max	4547	60.3222	7.4246	7.75e-05	9.87e-07	6.24e-07	1.37e-12
Ext non_negative	max	2605	58.6410	3.7906	2.21e-03	9.96e-07	5.95e-05	-6.87e-09
Ext all	max	1630	60.2208	2.4960	2.19e-03	9.98e-07	1.15e-04	-1.70e-09
N = 1000, K = 200								
Random	max	3552	75.6877	7.3998	1.25e-02	9.96e-07	5.14e-05	-1.78e-10
Internal	max	4412	73.1559	9.2662	5.99e-03	9.95e-07	5.96e-06	-2.72e-10
Ext sum_simplex	max	6185	74.2763	13.5116	4.24e-05	9.98e-07	2.79e-11	-1.30e-15
Ext non_negative	max	2104	73.5868	4.3128	1.73e-02	9.99e-07	2.55e-05	3.64e-14
Ext all	max	2966	73.9372	6.1266	1.80e-02	9.99e-07	8.30e-05	-9.74e-10

Table 2: Comparison of FW/AFW across different n and K

5.4 Impact of the spectral radius

In this section, we investigate how the spectral radius of the matrix Q affects the convergence behavior of FW and AFW.

To isolate the effect of $\rho(Q)$, we fixed the following parameters:

- Problem dimension: $n = 100$
- Number of simplices: $K = 20$
- Minimum eigenvalue of $Q = 1$
- Density = 1.0 (fully dense matrix)
- Kernel dimension = 0

We tested five values of the spectral radius: $\rho(Q) \in \{2, 10, 50, 100, 200\}$. As in previous experiments, we preserved the variation of the q -generation strategy across all configurations.

The stopping criterion was based on a F-W gap tolerance of $\epsilon = 10^{-6}$, with a maximum number of iterations set to 50000.

Table 3 reports the performance of FW and AFW for increasing values of spectral radius $\rho(Q)$, across all q -generation strategies.

From a general perspective, increasing the spectral radius negatively affects the performance of FW, which not only fails to converge in several configurations but, when it does, requires a substantially higher number of iterations. AFW is also influenced by higher curvature, but its convergence remains consistently faster across all tested values of $\rho(Q)$.

FW is able to converge in most cases where the sum simplex constraint is violated, except when the spectral radius becomes particularly large (i.e., $\rho = 200$). For AFW, we observe a moderate degradation in terms of iteration count starting from $\rho = 50$, particularly in the `internal`, `sum_simplex`, and `non_negative` configurations. In contrast, the `random` and `all` violation scenarios show stable performance, with the number of iterations remaining nearly unchanged as ρ increases.

In terms of the relative gap, AFW systematically provides better approximations of the optimum, confirming its advantage under ill-conditioned settings.

Figure 3 shows the convergence behavior for the `sum_simplex` violation, under two representative values of spectral radius for which FW also converges.

Subfigure 3a corresponds to $\rho = 2$, a relatively well-conditioned case. Here, FW and AFW both manage to converge, although FW exhibits slower progress and a more gradual reduction in the F-W gap. AFW achieves faster convergence with a steep and stable decrease in the early iterations.

Subfigure 3b refers to $\rho = 100$. In this setting, despite the ill-conditioned nature of the problem, FW does not fail to converge and successfully reaches the desired F-W gap within the maximum number of allowed iterations.

	Iterations		Time [s]		F-W Gap		Relative Gap	
	FW	AFW	FW	AFW	FW	AFW	FW	AFW
$\rho(Q) = 2$								
Random	31436	329	2.6315	0.0538	9.89e-07	9.97e-07	3.69e-12	-1.01e-13
Internal	max	191	4.4553	0.0374	4.01e-04	9.57e-07	1.22e-05	-8.62e-11
Ext sum_simplex	1917	225	0.2115	0.0540	9.93e-07	9.39e-07	1.92e-13	2.26e-14
Ext non_negative	max	250	4.2710	0.0400	8.97e-04	9.62e-07	2.98e-05	-1.88e-11
Ext all	max	232	4.2007	0.0376	9.28e-04	9.79e-07	9.96e-05	-6.20e-09
$\rho(Q) = 10$								
Random	max	569	4.5922	0.0948	3.79e-03	9.91e-07	9.96e-05	-7.88e-13
Internal	max	294	4.3943	0.0647	1.44e-03	9.83e-07	1.26e-05	-1.51e-09
Ext sum_simplex	5423	455	0.4706	0.1111	9.80e-07	9.83e-07	2.88e-14	0.00e+00
Ext non_negative	max	301	4.3489	0.0518	3.18e-03	9.92e-07	2.73e-05	-1.58e-10
Ext all	max	599	4.3596	0.1022	3.53e-03	9.49e-07	9.50e-05	-1.67e-09
$\rho(Q) = 50$								
Random	max	699	4.7105	0.1117	1.63e-02	9.20e-07	1.15e-04	-3.55e-12
Internal	max	826	4.3622	0.1330	6.74e-03	9.67e-07	1.23e-05	-3.02e-10
Ext sum_simplex	20814	1202	1.9212	0.1893	9.98e-07	9.96e-07	7.31e-15	1.83e-15
Ext non_negative	max	934	4.1950	0.1460	1.63e-02	9.88e-07	2.55e-05	-7.85e-12
Ext all	max	416	4.4096	0.0617	1.67e-02	9.94e-07	8.57e-05	-2.20e-09
$\rho(Q) = 100$								
Random	max	613	4.3093	0.1141	3.30e-02	9.35e-07	1.17e-04	-3.58e-09
Internal	max	1988	4.3680	0.3212	1.38e-02	9.99e-07	1.22e-05	-1.51e-10
Ext sum_simplex	36475	1314	3.7019	0.2128	9.92e-07	9.99e-07	2.24e-15	2.63e-16
Ext non_negative	max	1006	4.2517	0.1528	2.97e-02	9.97e-07	2.52e-05	2.35e-13
Ext all	max	410	4.2457	0.0628	3.12e-02	9.68e-07	8.41e-05	-2.77e-10
$\rho(Q) = 200$								
Random	max	679	4.2795	0.1174	5.95e-02	9.80e-07	1.18e-04	2.39e-12
Internal	max	2214	4.4340	0.3243	2.49e-02	9.90e-07	1.22e-05	-2.19e-09
Ext sum_simplex	max	1370	4.3931	0.2191	3.48e-06	9.89e-07	1.22e-14	3.83e-15
Ext non_negative	max	1070	4.3970	0.1906	6.15e-02	9.95e-07	2.50e-05	-2.41e-12
Ext all	max	441	4.9212	0.0857	5.93e-02	9.47e-07	8.34e-05	-9.14e-11

Table 3: Comparison of FW/AFW across different $\rho(Q)$

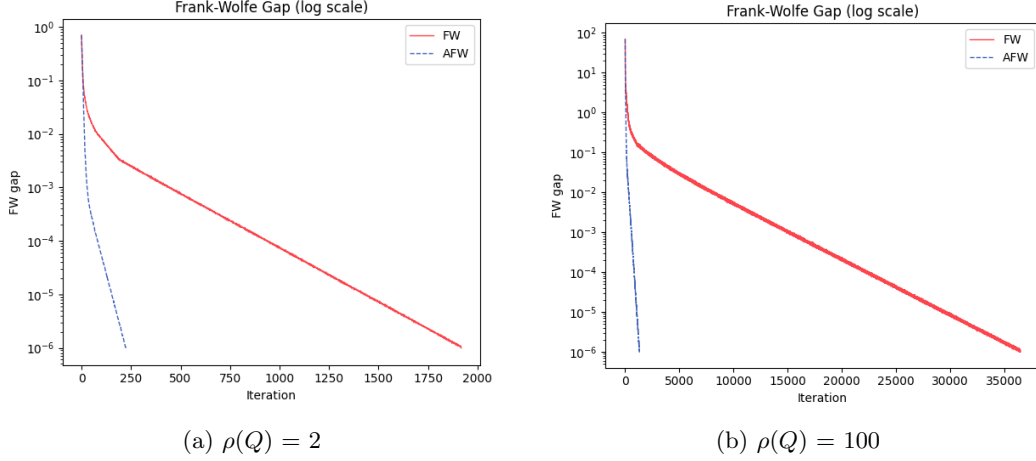


Figure 3: Convergence curves for sum simplex violation

5.5 Impact of the Null space

In this section, we analyze how the presence of a null space in the matrix Q affects the convergence behavior

To isolate the effect of $\dim_{ker}(Q)$, we fixed the following parameters:

- Problem dimension: $n = 100$
- Number of simplices: $K = 20$
- Spectral radius of $Q = 5$
- Minimum eigenvalue of $Q = 1$
- Density = 1.0 (fully dense matrix)

We tested four values of null space dimension: $\dim_{ker}(Q) \in \{20, 40, 60, 80\}$, progressively increasing the number of directions in which the quadratic form is linear.

The stopping criterion was based on a F-W gap tolerance of $\epsilon = 10^{-6}$, with a maximum number of iterations set to 50000.

We can clearly see from Table 4 that the performance of both FW and AFW generally improves as $\dim_{ker}(Q)$ increases. For FW, considering $\dim_{ker}(Q) = 80$, we observe a significant improvement in the **internal** and **ext.sum.simplex** configurations, and an even better results in the remaining violation scenarios. However, FW still fails to converge in the **random** case across all values of $\dim_{ker}(Q)$.

Regarding AFW, the results for kernel dimension of 20, 60 and 80 show its worst convergence performance in this experiment, respectively for **internal**, **all** and **random** settings. Notably, the **random** configuration shows a fluctuating trend, becoming the most demanding scenario when $\dim_{ker}(Q) = 80$.

	Iterations		Time [s]		F-W Gap		Relative Gap	
	FW	AFW	FW	AFW	FW	AFW	FW	AFW
<i>dim_ker(Q) = 20</i>								
Random	max	1013	4.1807	0.2113	1.47e-03	9.50e-07	1.48e-04	-1.65e-10
Internal	max	max	4.0867	7.2615	3.87e-04	3.49e-05	4.81e-06	3.00e-08
Ext sum_simplex	max	1512	4.2805	0.2377	1.53e-03	9.96e-07	7.26e-05	-1.80e-11
Ext non_negative	max	572	4.2784	0.0872	1.14e-03	9.40e-07	2.16e-05	-7.51e-10
Ext all	max	524	4.1826	0.0785	1.38e-03	9.59e-07	6.54e-05	-5.61e-10
<i>dim_ker(Q) = 40</i>								
Random	max	2341	4.2864	0.3440	1.21e-03	9.41e-07	4.19e-04	-8.35e-11
Internal	2205	677	0.1956	0.1092	9.90e-07	9.81e-07	9.86e-14	1.57e-14
Ext sum_simplex	4079	426	0.4174	0.0985	9.94e-07	9.31e-07	1.66e-13	-2.61e-14
Ext non_negative	max	602	5.1210	0.1185	9.00e-04	9.63e-07	1.53e-05	-1.82e-11
Ext all	max	713	4.3465	0.1326	9.19e-04	9.83e-07	4.70e-05	-4.62e-09
<i>dim_ker(Q) = 60</i>								
Random	max	1522	4.4687	0.2446	5.28e-04	9.61e-07	4.33e-04	-5.72e-10
Internal	230	133	0.0293	0.0284	9.26e-07	9.97e-07	1.99e-14	7.98e-15
Ext sum_simplex	311	123	0.0526	0.0269	9.34e-07	9.25e-07	2.67e-14	7.99e-15
Ext non_negative	max	3036	4.3170	0.4278	6.00e-04	9.81e-07	1.10e-05	-6.63e-11
Ext all	max	11513	4.2544	1.7827	7.09e-04	9.59e-07	4.95e-05	-5.32e-13
<i>dim_ker(Q) = 80</i>								
Random	max	4786	4.2808	0.6921	2.39e-04	9.88e-07	7.72e-05	-2.54e-10
Internal	51	44	0.0050	0.0079	7.61e-07	8.65e-07	1.02e-14	9.40e-15
Ext sum_simplex	79	49	0.0123	0.0109	7.46e-07	8.05e-07	1.02e-14	5.17e-15
Ext non_negative	576	281	0.1023	0.0501	9.90e-07	9.76e-07	5.07e-14	2.43e-14
Ext all	218	89	0.0235	0.0198	9.90e-07	9.49e-07	2.37e-14	-8.90e-15

Table 4: Comparison of FW/AFW across different $\dim_{\ker}(Q)$

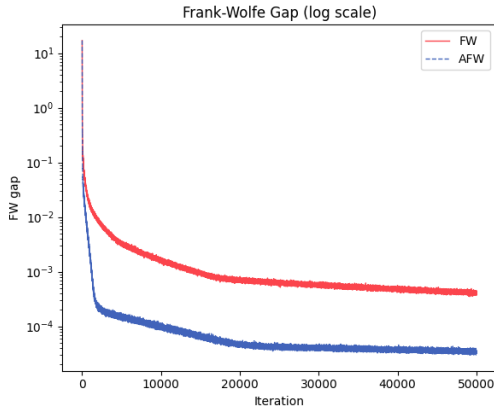


Figure 4 focuses on the **internal** configuration with $\dim_{\ker}(Q) = 20$. In this case, both FW and AFW fails to converge within the iteration limit, showing a sub-linear convergence behavior.

Figure 4: Convergence curves for `q-generation = internal` and `dim.ker(Q) = 20`

Figure 5 reports the convergence curves for all q -generation strategies under $\dim_{\ker}(Q) = 80$, the extreme case tested. We can clearly observe the sublinear convergence of FW in the **random** setting, along with a slower convergence rate for AFW in the same case. However, all the other configurations show similar convergence behavior for both FW and AFW, with steep descent and nearly overlapping curves, indicating fast and stable progress with the presence of a large null space.

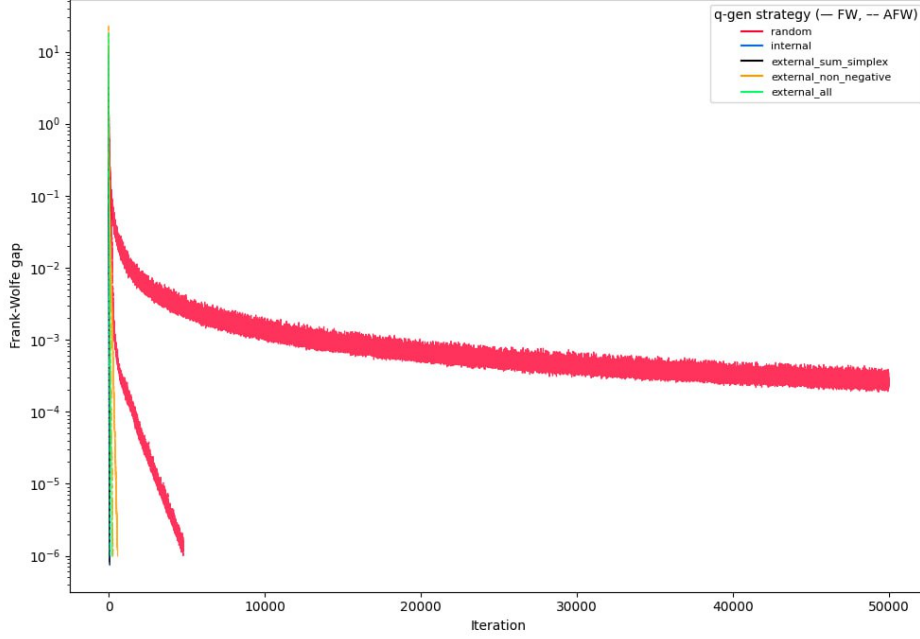


Figure 5: Convergence curves for q -generation strategies and $\dim_{\ker}(Q) = 80$

5.6 General purpose solver Gurobi

In this section, we compare the performance of the AFW algorithm with that of a general-purpose solver, specifically Gurobi.

Gurobi is used here as a benchmark for evaluating the quality of the solutions and the convergence efficiency of AFW. The results are reported in terms of iterations, execution time, F-W gap and relative gap only for AFW solutions.

We consider two experimental configurations:

- **Standard configuration:** the same adopted in the q -generation experiment (see section 5.2).
- **Ill-conditioned configuration:** characterized by a significantly larger problem size, high spectral radius, and a non-trivial null space dimension, resulting in a more challenging optimization setting.

5.6.1 Standard configuration

The configuration used in this experiment replicates the one adopted for the q -generation analysis. For clarity, the parameters are:

- Problem dimension: $n = 100$
- Number of simplices: $K = 20$
- Spectral radius of $Q = 5$
- Minimum eigenvalue of $Q = 1$
- Density = 1.0 (fully dense matrix)
- Kernel dimension = 0

As can be seen in Table 5, the Gurobi solver outperforms AFW in terms of iteration count. However, it is important to note that AFW generally requires less execution time, with particularly similar values observed in the **random** setting.

	Iterations		Time [s]		F-W Gap	Relative Gap
	AFW	Gurobi	AFW	Gurobi	AFW	AFW
Random	590	9	0.1045	0.1084	9.94e-07	-1.20e-09
Internal	230	12	0.0365	0.1083	9.75e-07	-1.72e-09
Ext sum_simplex	317	8	0.0545	0.1081	9.73e-07	7.02e-15
Ext non_negative	298	12	0.0623	0.1091	9.34e-07	-3.01e-09
Ext all	542	10	0.0855	0.1083	9.70e-07	-1.01e-11

Table 5: AFW vs Gurobi standard configuration

5.6.2 Ill-conditioned configuration

In this second experiment, we test the algorithms under more challenging conditions. We fixed the following parameters:

- Problem dimension: $n = 1000$

- Spectral radius of $Q = 500$
- Minimum eigenvalue of $Q = 1$
- Density = 1.0 (fully dense matrix)
- Kernel dimension: 20

We tested two values for the number of partitions: $K \in \{20, 200\}$.

As can be seen from Table 6, varying the number of partitions K results in differences in the performance of AFW across the first four q -generation strategies. In contrast, Gurobi maintains stable behavior both in terms of iteration count and execution time. AFW outperforms Gurobi in terms of execution time in certain cases, such as when $K = 20$ and the q -generation strategy is **non_negative**. However, due to the increased difficulty of the problem, Gurobi better deals with the ill-conditioned setting overall, showing more stability in terms of the convergence speed.

	Iterations		Time [s]		F-W Gap	Relative Gap
	AFW	Gurobi	AFW	Gurobi	AFW	AFW
K = 20						
Random	9002	11	16.2158	10.3984	9.82e-07	-1.06e-09
Internal	max	11	89.4179	10.4455	2.11e-03	8.68e-08
Ext sum_simplex	42759	10	76.0166	10.3724	9.97e-07	-1.13e-10
Ext non_negative	2846	12	4.4859	10.4765	9.99e-07	-1.09e-09
Ext all	2288	11	3.7947	10.4082	9.93e-07	-2.42e-09
K = 200						
Random	4275	10	9.8752	10.3913	9.88e-07	-2.37e-10
Internal	9959	11	23.6377	10.3920	9.94e-07	-2.84e-11
Ext sum_simplex	9176	8	22.0125	10.3076	9.88e-07	-4.79e-12
Ext non_negative	8295	11	19.1838	10.4142	9.99e-07	-1.12e-09
Ext all	2753	11	6.4436	10.5791	9.83e-07	-1.06e-09

Table 6: AFW vs Gurobi ill-conditioned configuration

5.7 General discussion

The experimental results confirmed the theoretical advantages of AFW. Across all configurations, AFW consistently outperformed vanilla FW in terms of convergence speed and solution quality. The option to perform away steps was important to deal with the zig-zagging behavior that limits FW. Different experiments show how the structure and conditioning of the problem affect convergence. While increasing the number of variables and simplices led to a reduction in performance, FW suffered significantly more than AFW. High spectral radius and non-zero null spaces further show FW’s limitations. AFW still performed well, although some drop in performance was also observed under extreme conditions. The comparison with Gurobi demonstrated that AFW achieved comparable or even better performance in terms of execution time, although Gurobi maintained consistency.

References

- [1] Immanuel M. Bomze, Francesco Rinaldi, and Damiano Zeffiro. *Active set complexity of the Away-step Frank-Wolfe Algorithm*. 2019. arXiv: 1912.11492 [math.OC]. URL: <https://arxiv.org/abs/1912.11492>.
- [2] Immanuel. M. Bomze, Francesco Rinaldi, and Damiano Zeffiro. *Frank-Wolfe and friends: a journey into projection-free first-order optimization methods*. 2021. arXiv: 2106.10261 [math.OC]. URL: <https://arxiv.org/abs/2106.10261>.
- [3] Vandenberghe L. Boyd S. *Convex optimization*. Cambridge university press., 2008.
- [4] Gábor Braun et al. *Conditional Gradient Methods*. 2023. arXiv: 2211.14103 [math.OC]. URL: <https://arxiv.org/abs/2211.14103>.
- [5] Sébastien Bubeck. “Convex Optimization: Algorithms and Complexity.” In: *Foundations and Trends in Machine Learning*. 8, 3-4 (2015), pp. 231–358.
- [6] NumPy Developers. *numpy.linalg*. Accessed: 17 March 2025. 2024. URL: <https://numpy.org/doc/stable/reference/generated/numpy.linalg.html>.
- [7] Marguerite Frank, Philip Wolfe, et al. “An algorithm for quadratic programming”. In: *Naval research logistics quarterly* 3.1-2 (1956), pp. 95–110.
- [8] S.J. Wright J. Nocedal. *Numerical Optimization – second edition*. Springer Series in Operations Research and Financial Engineering, 2006.
- [9] Simon Lacoste-Julien and Martin Jaggi. “On the global linear convergence of Frank-Wolfe optimization variants”. In: *Advances in neural information processing systems* 28 (2015).
- [10] Javier Pena and Daniel Rodriguez. “Polytope conditioning and linear convergence of the Frank–Wolfe algorithm”. In: *Mathematics of Operations Research* 44.1 (2019), pp. 1–18.
- [11] Sebastian Pokutta. *The Frank-Wolfe algorithm: a short introduction*. 2023. arXiv: 2311.05313 [math.OC]. URL: <https://arxiv.org/abs/2311.05313>.